

Guía introductoria del proyecto **ai-specs**

Modelo SDD (Spec-Driven Development) como flujo medular del ciclo de vida del software

1) Propósito de esta guía

Este documento explica de forma integral cómo funciona el proyecto **ai-specs** y cómo aplicar **SDD (Spec-Driven Development)** como práctica central de trabajo para equipos humanos y agentes de IA.

El objetivo principal de **ai-specs** es convertir una necesidad de negocio en software trazable, verificable y mantenible, evitando ambigüedades y decisiones implícitas.

2) Qué es **ai-specs** y por qué existe

ai-specs es un marco de trabajo que define:

- Estándares base de desarrollo (calidad, idioma, tipo de tickets, seguridad, testing).
- Flujo operativo de punta a punta desde discovery hasta pull request.
- Artefactos normativos (PRD, open questions, UX flows, ADRs, planes de implementación).
- Comandos operativos para ejecutar fases específicas del ciclo SDD.

En lugar de “codificar primero y aclarar después”, el proyecto establece el principio:

Primero especificar, después validar, luego implementar.

3) Principios fundacionales del modelo

El proyecto se apoya en principios no negociables:

- **Small steps**: cambios pequeños, una cosa a la vez.
- **TDD**: nuevas funcionalidades deben iniciar con tests que fallen.
- **Type safety**: código tipado y contratos explícitos.
- **Naming clarity**: nombres semánticos y consistentes.
- **Incremental delivery**: iteraciones acotadas y verificables.
- **No assumptions**: si falta información, se pregunta o se marca **TBD**.
- **Refinement over duplication**: cada artefacto derivado debe añadir información verificable nueva.

Estos principios reducen retrabajo, mejoran predictibilidad y elevan la calidad de entrega.

4) SDD en una frase

SDD es un sistema de decisiones guiadas por especificación, con gates de calidad y trazabilidad obligatoria desde requisito hasta código.

5) Flujo medular SDD (Fases 0 a 8)

Fase 0 - Discovery Gate (obligatoria)

Antes de escribir el PRD se hacen preguntas críticas al stakeholder (mínimo dos) sobre:

- problema núcleo,
- métricas de éxito,
- restricciones,
- alcance y no alcance.

Regla clave: **no iniciar PRD con supuestos.**

Fase 1 - PRD (fuente única de verdad)

Artefacto principal: `document/PRD.md`.

Debe incluir:

- Executive Summary,
- User Experience & Functionality (con user stories e acceptance criteria),
- Technical Specifications,
- Risks & Roadmap.

Complemento: `document/PRD-analisis-critico.md` para detectar ambigüedades y inconsistencias.

Fase 2 - Gate-PRD

Antes de crear tickets o artefactos derivados, el PRD debe pasar un gate que valida:

- semántica unívoca de reglas de negocio,
- estructura esperada de resultados,
- manejo de “no aplica”,

- manejo de errores,
- idempotencia,
- estados y transiciones.

Si falla, se corrige PRD y/o se escalan dudas a [document/open-questions.md](#).

Fase 3 - Decisión de artefactos (modo mínimo vs completo)

El modo por defecto es **mínimo**.

Solo se crean artefactos adicionales cuando aportan claridad verificable:

- **UX Flow** si hay ramas/estados complejos.
- **ADR** si hay decisión arquitectónica no trivial.
- **US refinada separada** si los AC del PRD no alcanzan para mapear pruebas sin reinterpretar.

Fase 4 - Crear ticket en Redmine ([/create-us](#))

Se crea ticket solo después del Gate-PRD.

El ticket nace en estado [Design](#) y debe contener:

- clasificación,
- historia de usuario,
- criterios de aceptación verificables (incluyendo error),
- notas técnicas y riesgos,
- trazabilidad al PRD.

Fase 5 - Enriquecer ticket ([/enrich-us](#))

Se ejecuta Mini Gate-PRD a nivel ticket:

- AC verificables,
- errores/validaciones explícitos,
- idempotencia cuando aplique,
- diseño de referencia resuelto si existe UI.

Si hay bloqueos, permanece en [Design](#) y se documenta por qué.

Fase 6 - Gate-final

Valida integridad del paquete completo:

- relaciones PRD ↔ artefactos ↔ ticket sin contradicción,
- mapeo AC → pruebas/contratos,
- evidencia de que UX/ADR añaden información nueva.

Con gate aprobado, el ticket puede pasar a `Development`.

Fase 7 - Planificación de implementación

Con ticket en `Development`:

- backend: `/plan-backend-ticket` → `ai-specs/changes/[id]_backend.md`
- frontend: `/plan-frontend-ticket` → `ai-specs/changes/[id]_frontend.md`

Aquí se define ejecución paso a paso para implementación.

Fase 8 - Implementación y PR

Comandos:

- `/develop-backend`
- `/develop-frontend`

Se implementa con tests, lint y type checking, branch dedicada, commit descriptivo y PR enlazado al ticket.

6) Gates de calidad y por qué son críticos

Los gates son el mecanismo de control de riesgo del sistema:

- Detectan ambigüedad temprano.
- Evitan propagar errores semánticos al código.
- Fuerzan trazabilidad real entre requisitos, ticket e implementación.
- Reducen bloqueos tardíos en QA/PR.

Si un gate falla, el proceso se detiene y se corrige en origen (normalmente PRD o ticket).

7) Trazabilidad end-to-end

En `ai-specs`, todo cambio debe responder:

1. ¿Qué necesidad de negocio atiende?
2. ¿Dónde está especificado?

3. ¿Qué ticket lo ejecuta?
4. ¿Qué pruebas lo validan?
5. ¿Qué PR lo materializa?

Cadena esperada:

Discovery -> PRD -> (UX/ADR/US refinada si aplica) -> Redmine ticket -> Plan -> Código + Tests
-> PR

8) Reglas de idioma y colaboración

- **Artefactos técnicos:** inglés (código, nombres técnicos, API docs, tests).
- **Narrativa operativa y documentación:** español.
- **Tickets Redmine:** español.

Esto permite consistencia técnica sin perder claridad para stakeholders hispanohablantes.

9) Redmine dentro del flujo SDD

El ticket no reemplaza al PRD; lo operacionaliza.

Regla de clasificación:

- valor funcional de negocio -> **Feature**
- implementación técnica derivada -> **Task**

Estado inicial recomendado: **Design**, y transición a **Development** solo tras gates aprobados.

10) Artefactos principales y ubicación

- PRD: **document/PRD.md**
 - Análisis crítico: **document/PRD-analisis-critico.md**
 - Open Questions: **document/open-questions.md**
 - UX Flows: **document/ux-flows/**
 - ADRs: **document/adrs/**
 - Planes de implementación: **ai-specs/changes/**
 - Comandos del flujo: **ai-specs/.commands/**
-

11) Anti-patrones que el modelo evita

- Crear tickets sin Gate-PRD.
- Duplicar contenido del PRD en cada artefacto.
- Generar UX/ADR sin trigger real.
- Implementar con TBDs críticos sin resolver.
- Pasar a desarrollo con diseño UI pendiente (**Diseño Ref: TBD**).
- Hacer PR sin pruebas ni trazabilidad.

Estos anti-patrones son causas frecuentes de retrabajo y deuda semántica.

12) Rol de AI Engineers y agentes en este proyecto

Un AI engineer en **ai-specs** no solo "genera texto/código"; también:

- orquesta el flujo por fases,
- valida gates y calidad de requisitos,
- protege trazabilidad,
- convierte incertidumbre en preguntas estructuradas,
- evita decisiones implícitas.

El valor diferencial está en mantener disciplina de proceso, no solo velocidad de ejecución.

13) Beneficios esperados al adoptar SDD plenamente

- Mejor alineación negocio-tecnología.
 - Menos retrabajo por requisitos ambiguos.
 - Mayor precisión en estimaciones.
 - Entregas más auditables y reproducibles.
 - Reducción de defectos por interpretación inconsistente.
 - Mejor coordinación entre equipo humano y agentes de IA.
-

14) Recomendaciones de adopción para equipos nuevos

1. Entrenar al equipo en Discovery y Gate-PRD antes de hablar de herramientas.
2. Usar modo mínimo por defecto para evitar burocracia.
3. Activar modo completo solo con triggers justificados.

4. Revisar **open-questions** como ritual semanal.
 5. Medir salud del flujo (tiempos de Design -> Development, bloqueos por ambigüedad, defectos por AC incompletos).
-

15) Conclusión

ai-specs implementa SDD como un sistema operativo de producto e ingeniería: cada paso busca preservar claridad, verificabilidad y trazabilidad.

La regla más importante para equipo/AI es simple:

Si no está especificado y validado, no está listo para implementarse.